02456 - DEEP LEARNING PROJECT - GROUP 8

Ole Decker (S253728), Mikel García (S253702), Diego Medina (S253699)
Ivan Musalyk (S225210), Dhruva Sakhare (S225191)

ABSTRACT

Large-scale experimental mapping of cellular responses to genetic perturbations is costly and cannot cover the full combinatorial space of gene interactions. In this project, we develop a deep learning framework for in silico perturbation of single-cell transcriptomes using data from the Virtual Cell Challenge, where cells are represented in a 50-dimensional latent space learned by scVI. On this latent space, our conditional Flow Matching model learns deterministic trajectories from control to perturbed states, using gene embeddings to enable zero-shot predictions for unseen genes.

1. INTRODUCTION

Understanding cellular responses to genetic perturbations is fundamental to biology, yet experimentally mapping the vast combinatorial space of gene interactions using techniques like CRISPR interference and single-cell RNA sequencing (scRNA-seq) is prohibitively expensive. The Virtual Cell Challenge addresses this bottleneck with a dataset of over 183,000 single-cell profiles across approximately 150 gene knockdowns. We aim to bridge the gap between sparse experimental data and this vast interaction space by building a deep learning framework for *in silico* perturbations, predicting phenotypic outcomes for specific gene knockdowns including unseen perturbations, by learning a latent-space surrogate model of cellular responses.

To represent cells in a compact latent space suitable for flow-based modeling, we experimented with both the SE-600M embedding from the STATE framework [1] and the scVI model [2]. In our final approach, we model the transition from control to perturbed states using the Flow Matching framework on 50-dimensional scVI latent embeddings. We adopt Flow Matching instead of diffusion-based perturbation models because FM avoids stochastic sampling, produces smooth biologically meaningful trajectories, and enables faster inference for large-scale in-silico screening. Unlike diffusion models that generate data from noise via stochastic processes, Flow Matching defines a deterministic Ordinary Differential Equation (ODE) that continuously transforms a specific control cell embedding into its perturbed counterpart. Our implementation utilizes a VectorFlowNet, a residual architecture conditioned on gene embeddings via Adaptive Layer Normalization (AdaLN), to predict the velocity field driving these trajectories. This approach offers a biologically interpretable model of perturbation progression.

2. MATERIALS AND METHODS

Github Page:
`https://github.com/MikelGarciaData/DL_VCC`

2.1. Dataset description

The dataset used in this project was created by the Arc Institute for the Virtual Cell Challenge. [3]. It is the transcriptomic reference of unperturbed H1 human embryonic stem cells (hESCs). The dataset consisting of single-cell profiles for 150 gene perturbations (183,000 cells). The basic layout of the data file is shown in (table 1)

2.2. The SCVI Embedding

SCVI (Single-cell Variational Inference) is a deep generative model based on a Variational Autoencoder (VAE). It constructs a low-dimensional latent representation of cells from high-dimensional gene expression data.

The model takes as input the raw count values (restricted to the $9{,}500$ highly variable genes) together with the batch annotation.

The VAE encoder compresses the high-dimensional gene expression vector into a latent space of 50 dimensions. This latent vector is stored as `X_scvi`. These embeddings were used for training our model.

The resulting 50-dimensional representation captures the state of each cell in a denoised and biologically meaningful way.

2.3. Flow Matching

Flow matching is the core learning component of our model. It was recently introduced as an alternative to diffusion-based generative models. Whereas diffusion models learn to reverse a stochastic noising process, flow matching instead learns the *deterministic* straight paths (flows) between the data distribution and a prior distribution by parameterizing an ordinary differential equation (ODE).

Table 1. Gene Expression File in AnnData H5AD format

cell barcode–batch index	target_gene	guide_id	batch
...AAGCAACCTTGTACTTTAGG-Flex_1_01	CHMP3	CHMP3_P1P2_A — CHMP3_P1P2_B	Flex_1_01
...GACGTGGTGCAGATTCGGTT-Flex_3_16	non-targeting	non-targeting_00035 — non-targeting_03439	Flex_3_16

In our project, the model is trained by minimizing the flow-matching loss, as described in Algorithm 1. The algorithm is a modified version of the procedure introduced in *Holderrieth et al. (2025)*.

Algorithm 1 Training Procedure for Flow Matching

Require: A dataset of pairs $(x_{\text{ctrl}}, x_{\text{pert}})$; perturbed gene embedding y_vec

1: **for** each mini-batch of data **do**
2: Sample control cells x_{ctrl} and perturbed cells x_{pert}
3: Get gene condition vector y_vec
4: Sample random time $t \sim \text{Uniform}(0, 1)$
5: Compute cell state at time t

$$x_t \leftarrow (1-t)x_{\text{ctrl}} + t x_{\text{pert}}$$

6: Calculate velocity

$$u_{\text{target}} \leftarrow x_{\text{pert}} - x_{\text{ctrl}}$$

7: Predict velocity

$$u_{\text{pred}} \leftarrow \text{Model}(x_t, t, y_{vec})$$

8: Calculate loss

$$\mathcal{L} = \|u_{\text{pred}} - u_{\text{target}}\|^2$$

9: **end for**

2.4. Model Conditioning

Our model learns a velocity field

$$v(x, t, y),$$

describing how a cell evolves from a control state to a perturbed state over continuous time t. Conditioning is provided through two inputs: the time variable t and a gene vector y_{vec}.

2.4.1. Condition Embeddings

Each conditioning input (time, perturbation gene) is embedded separately before being used by the network.

The scalar time value is mapped into a high-dimensional representation using sinusoidal positional embeddings followed by a small MLP. This provides the model with a rich and smoothly varying encoding of where a cell lies along the trajectory.

The perturbation genes in the dataset are just gene names and give no information about the gene and how it functions. So we embedded the gene names into a continuous vector using *genePT*[4] embeddings that encodes biological meaning to the gene vectors. This allows the model to predict perturbation of genes that were not present in the training set but are similar in function.

2.4.2. Conditioning via AdaLN

The two embeddings are concatenated to form a unified conditioning vector. This vector is applied to the main network through *Adaptive Layer Normalization* (AdaLN), which generates learned scale and shift parameters. Given input x and condition c, AdaLN computes:

$$\text{AdaLN}(x, c) = \text{LayerNorm}(x)\left(1 + \gamma_c\right) + \beta_c.$$

This mechanism enables time and gene identity to modulate the activations of every residual block.

2.5. Residual Blocks and Velocity Learning

The network learns the velocity field through a sequence of pre-activation residual blocks. Each block operates in three steps:

1. **Conditional Normalization.** AdaLN is applied first, shifting and scaling the cell representation based on the time t and the perturbation identity.
2. **Feature Transformation.** The conditioned features pass through a nonlinear transformation (an MLP), which learns the incremental change in the velocity field.
3. **Residual Update.** The transformed output is added back to the input of the block.

Overall, these conditioned residual blocks together learn a smooth, perturbation-specific velocity field describing cellular transitions over time.

2.6. VectorFlowNet

VectorFlowNet computes the velocity vector, which directs the ODE solver to move a cell in gene expression space toward its perturbed state at any time t.

The process consists of three main steps:

1. **Embedding and Fusion.** The cell x, time t, and gene vector y_{vec} are embedded and combined into a single context vector.

2. **Deep Processing.** The cell embedding passes through six residual blocks, which refine the representation using the context vector.

3. **Output.** The processed features are projected back to the original input dimension to produce the instantaneous velocity vector for the cell.

2.7. ODE Trajectory Simulation

The `ode_solve_trajectory` function simulates the evolution of a control cell under a perturbation over time $t \in [0, 1]$.

While the neural network computes the velocity vector, the ODE solver performs the actual integration. Euler's method is used to update the cell's state in discrete steps:

$$x_{\text{new}} = x_{\text{old}} + v \cdot \Delta t,$$

where v is the velocity predicted by the model at the current state and Δt is the step size. This process is repeated until the final time $t = 1$, yielding the simulated trajectory of the cell under the perturbation.

2.8. Validation Split and Early Stopping

Gene-based Validation Split:

Instead of splitting cells randomly, the dataset is divided based on perturbed genes. All the cells with a specific perturbed gene are split into training and validation sets using `train_test_split` based on the perturbation:

- **Training set:** Cells corresponding to approximately 80% of the genes.
- **Validation set:** Cells corresponding to the remaining 20% of the genes.

This evaluates the model's ability to generalize to *unseen perturbations*, ensuring that validation assesses prediction accuracy for genes not observed during training.

Early Stopping:

During training, the validation loss is monitored. Training is stopped early if the validation loss does not improve, preventing overfitting and ensuring better generalization.

3. RESULTS

3.1. Training Dynamics and Effect of subsampling and model size

When trained on the SCVI latent space from the subset of a smaller number of cells, the model showed a stable and monotonic improvement in both training and validation loss (Figure 1). The validation loss decreased continuously and reached its minimum at epoch 189, indicating good convergence and generalization capacity.

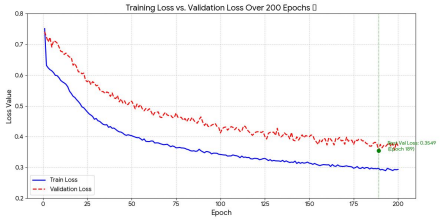

Fig. 1. Training and validation loss of the Flow Matching model trained on SCVI embeddings generated from a subset of 7000 cells randomly picked from the original trainingset.

the model exhibited a different behavior (Figure 2) when trained with the full dataset. Although the training loss decreased steadily, the validation loss plateaued at around 100 epochs.

When increasing the model complexety we see a slight improvement on the training and validation loss curve (figure 3) the validation loss goes down to around 1.3 but plateaues at around 50 epochs, earlier then in the smaller model.

Fig. 2. Flow Matching model with 128 hidden dimensions and 4 layersSCVI embedding dimensions.

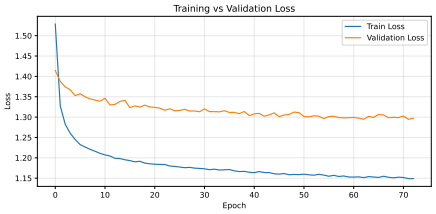

Fig. 3. Flow Matching model with 256 hidden dimensions and 8 layers trained on SCVI embedding dimensions

3.2. Trajectory Predictions of Gene Perturbations

To check whether our model is able to learn zero-shot perturbation dynamics, we simulated trajectories from $t = 0$ to

$t = 1$ for every gene in the validation set. We then projected the predicted cell states onto UMAP for visualisation (Fig. 4). (Fig. 5).

At $t = 0$, all predictions originate from the control population. As time progresses, the predicted trajectories gradually diverge from this control manifold, illustrating the evolution of cell states over time. The final results do not display distinct cell clusters, likely because most genes do not induce substantial changes in cell states. Additionally, the limited size of the training dataset restricted the model's ability to capture the effects of many gene perturbations. However, cells perturbed with **SMARCA5** (highlighted in pink) exhibit clear differentiation, suggesting that the model successfully learned to predict cell state changes for this specific perturbation.

We then looked the UMAP of the true dataset and found SMARCA5 to form a distinct cluster so we we made an UMAP with only the SMARCA5 predictions and the control cells to see if the model differentiates the cells.(Fig. 8)

We can see that the SMARCA5 forms a distinct cluster outside of the control cells in (Fig. 8)

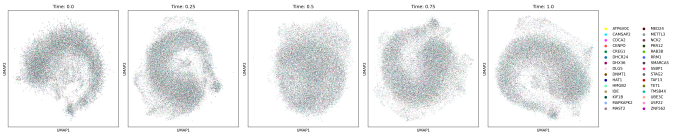

Fig. 4. UMAP projection of predicted trajectories generated by the model trained on 128 hidden dimension.

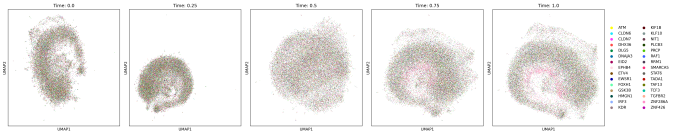

Fig. 5. Description UMAP projection of predicted trajectories generated by the model trained on 128 hidden dimension.

Fig. 6. UMAP projection of actual perturbed genes.

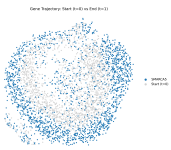

Fig. 7. UMAP projection of SMARCA5 predictions trained on model with 128 dimensions.

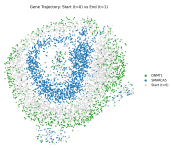

Fig. 8. UMAP projection of SMARCA5 and DNMT1 predictions trained on model with 256 hidden dimensions.

4. DISCUSSION

4.1. Limitations on the model evaluation

One aspect that we must note in this work is that no independent test set was used. Only the training and validation sets were used. This leads us to a metodological limitation, since we do not have a set with which to test the model's performance, the validation set is used both to adjust hyperparameters and to check the model's performance. Therefore, our estimates of the model's performance may be somewhat optimistic.

We initially experimented with SE-600M embeddings, but training was unstable and did not converge to meaningful solutions. SE embeddings are extremely high-dimensional which made it difficult to calculate the loss.

4.2. Model improvements for future iterations

MLP alone may not be sufficient to capture the relationship between genes and cell states. Deeper architectures or attention-based architectures might have yielded better results, even though they are more complex to implement.

Differential gene expression analysis would be good as an additional validation method, as it would let us verify whether the model's predicted perturbation effects align with known gene-level expression changes.

DECLARATION OF USE OF GENERATIVE AI

- We have used generative AI tools: yes

List the generative AI tools you have used:

- ChatGPT 5.1
- GitHub Copilot (model Gemini 2.5 Pro)

What did you use the tool(s) for? The tools were used for code debugging, exploring alternative implementation approaches, and performing high-level reasoning checks during model development.

At what stage(s) of the process did you use the tool(s)? They were used throughout development, including during coding, model setup, and refinement.

How did you use or incorporate the generated output? All AI-generated suggestions were manually reviewed, verified for correctness, and adapted before use. Core research ideas, analysis, and conclusions were produced independently by the authors.

References

[1] A. K. Adduri et al., "Predicting cellular responses to perturbation across diverse contexts with state," *bioRxiv preprint*, 2025. DOI: 10.1101/2025.06.26.661135.

[2] R. Lopez, J. Regier, M. B. Cole, M. I. Jordan, and N. Yosef, "Deep generative modeling for single-cell transcriptomics," *Nature Methods*, vol. 15, pp. 1053–1058, 2018. DOI: 10.1038/s41592-018-0229-2.

[3] Y. H. Roohani et al., "Virtual Cell Challenge: Toward a Turing test for the virtual cell," *Cell*, vol. 188, no. 13, pp. 3370–3374, Jun. 26, 2025, ISSN: 0092-8674, 1097-4172. DOI: 10.1016/j.cell.2025.06.008. PMID: 40578317. Accessed: Dec. 5, 2025. [Online]. Available: https://www.cell.com/cell/abstract/S0092-8674(25)00675-0.

[4] Y. T. Chen and J. Zou, "Genept: A simple but effective foundation model for genes and cells built from chatgpt," *bioRxiv*, 2023. DOI: 10.1101/2023.10.16.562533.